MLDL
PRACTICAL 5

Name: Payal Dara
Class: D15C
Roll no: 56
Batch: C

AIM:-

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source

Dataset Name: Bank Marketing Dataset
Source: UCI Machine Learning Repository
Dataset Link:
https://archive.ics.uci.edu/ml/datasets/Bank+Marketing

Data Collected By: Portuguese Banking Institution
Data Collection Method: Direct marketing campaigns (phone calls)

---

2. Dataset Description

The Bank Marketing dataset contains information about marketing campaigns conducted by a bank to predict whether a client will subscribe to a term deposit.

Dataset Characteristics

- Number of Instances: 45,211
- Number of Features: 16 input features
- Target Variable: y (binary: yes/no)

Target Variable

- 1 → Client subscribed
- 0 → Client did not subscribe

Feature Types

- **Categorical:** Job, Marital, Education, Contact, etc.
- **Numerical:** Age, Balance, Duration, Campaign, etc.

Observations

- Imbalanced dataset (~88% no, ~12% yes)
- Mix of categorical and numerical features
- Requires encoding and scaling

---

3. Mathematical Formulation of SVM

Support Vector Machine is a supervised learning algorithm used for classification. It finds the optimal hyperplane that separates data into classes.

Hyperplane Equation

$w \cdot x+b=0$$w \cdot x+b=0$

Where:

- w = weight vector
- x = input features
- b = bias

Objective Function

$\min \frac{1}{2}||w||^2$$\min \frac{1}{2}||w||^2$

Subject to:

$y_i(w \cdot x_i+b) \geq 1$$y_i(w \cdot x_i+b) \geq 1$

Soft Margin (with regularization)

min12||w||2+C∑ξimin21||w||2+C∑ξi

Where:

- CC = regularization parameter
- ξiξi = slack variables (allow misclassification)

---

4. Algorithm Limitations (SVM)

- Computationally expensive for large datasets
- Sensitive to choice of kernel and parameters
- Difficult to interpret compared to Logistic Regression
- Requires proper feature scaling
- Performance drops with noisy data

---

5. Methodology / Workflow

Steps:

1. Load the dataset
2. Convert target variable (yes/no → 1/0)
3. Handle missing values
4. Encode categorical variables (One-Hot Encoding)
5. Scale numerical features (Standardization)
6. Split dataset into training and testing sets
7. Train SVM classifier
8. Perform hyperparameter tuning
9. Evaluate model performance

Workflow Diagram (Textual)

Dataset → Preprocessing → Encoding → Scaling → Train-Test Split
→ SVM Training → Hyperparameter Tuning → Prediction → Evaluation

---

6. Performance Analysis

Evaluation Metrics Used

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC-AUC Score

Interpretation

- Accuracy may be misleading due to class imbalance
- Precision shows correctness of positive predictions
- Recall shows how many actual positives are detected
- F1-score balances precision and recall
- ROC-AUC measures classification ability

---

7. Hyperparameter Tuning (SVM)

Key Parameters

- **C:** Controls regularization (higher C = less margin, more overfitting)
- **Kernel:** Defines transformation (linear, rbf, poly)
- **Gamma:** Controls influence of data points (used in RBF kernel)

CODE:-

```
# ==============================
```

```
# STEP 1: Import Libraries

# ================================

import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report


# ================================

# STEP 2: Load Dataset (Online)

# ================================

url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/banknote_authentica
tion.csv"

df = pd.read_csv(url, header=None)


# Add column names

df.columns = ['variance', 'skewness', 'curtosis', 'entropy', 'class']

# ================================

# STEP 3: Preprocessing

# ================================
```

```
X = df.drop('class', axis=1)
y = df['class']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ================================
# STEP 4: SVM WITHOUT TUNING
# ================================
svm = SVC()
svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)
```

```
print("===== WITHOUT TUNING =====")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
print("Classification Report:\n", classification_report(y_test, y_pred_svm))

# ================================
# STEP 5: SVM WITH TUNING
# ================================
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

grid = GridSearchCV(SVC(), param_grid, cv=5)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_

y_pred_tuned = best_model.predict(X_test)

print("\n===== WITH TUNING =====")
```

```
print("Best Parameters:", grid.best_params_)

print("Accuracy:", accuracy_score(y_test, y_pred_tuned))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))

print("Classification Report:\n", classification_report(y_test, y_pred_tuned))
```

```
... ===== WITHOUT TUNING =====
    Accuracy: 1.0
    Confusion Matrix:
     [[148   0]
     [  0 127]]
    Classification Report:
                   precision    recall  f1-score   support

               0       1.00      1.00      1.00       148
               1       1.00      1.00      1.00       127

        accuracy                           1.00       275
       macro avg       1.00      1.00      1.00       275
    weighted avg       1.00      1.00      1.00       275


    ===== WITH TUNING =====
    Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
    Accuracy: 1.0
    Confusion Matrix:
     [[148   0]
     [  0 127]]
    Classification Report:
                   precision    recall  f1-score   support

               0       1.00      1.00      1.00       148
               1       1.00      1.00      1.00       127

        accuracy                           1.00       275
       macro avg       1.00      1.00      1.00       275
    weighted avg       1.00      1.00      1.00       275
```

Interpretation:

The accuracy before and after hyperparameter tuning remains almost the same, indicating that the default SVM parameters were already suitable for the dataset.

However, tuning helps in selecting optimal parameters and ensures better model reliability and generalization.

CONCLUSION:-

Support Vector Machine (SVM) is an effective classification algorithm that provides good accuracy for the given dataset. The model performs well even with default parameters, and hyperparameter tuning helps in selecting optimal values for better reliability. Overall, SVM is suitable for classification tasks and can achieve strong performance with proper parameter selection.